

Sistemas Operativos

Departamento de Computación - FCEyN - UBA
Primer cuatrimestre de 2025

Nombre y apellido: Pedro Fuente Urteaga
Nº orden: 44 L.U.: 1071/22 Cant. hojas: 4

Primer recuperatorio - 1er cuatrimestre de 2025

1	2	3	4	Nota
25	25	25	25	100

(A)

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas. Numere las hojas entregadas. Complete en la primera hoja la cantidad total de hojas entregadas. Entregue esta hoja junto al examen, la misma no se incluye en la cantidad total de hojas entregadas. La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente): 0 a 64 pts y A (Aprobado): 65 a 100 pts.

Ejercicio 1.(25 puntos)

Se desea implementar un Servidor para gestionar las actas de finales de la Universidad, utilizando sockets. Cada docente se conecta al Servidor para crear el acta de su materia (un sólo docente por materia), indicando un código numérico (int) que representa a la materia, y comienza a cargar notas. Por cada estudiante que rinde el final, el docente envía un mensaje indicando el número de libreta universitaria (strings con la forma NNNN/NN) y la nota final (int). Al finalizar la carga, el Servidor debe cerrar el acta y cerrar la conexión. El Servidor debe ser capaz de manejar simultáneamente N docentes cargando al mismo tiempo notas finales. Se dispone de una estructura llamada Acta, y de las siguientes funciones:

```
Acta crearActa(int codMateria); //Crea un acta para la materia con el código indicado
void agregarNota(Acta acta, char* lu, int nota); //Agrega nota de estud con lu al acta
void cerrarActa(Acta acta); //Cierra el acta
```

Completar el código del proceso Servidor y de uno de los procesos Docente.

```
int main_servidor() {
    int servidor_fd, cliente_fd;
    struct sockaddr_un direccion;
    socklen_t addrlen = sizeof(direccion);
    direccion.sin_family = AF_UNIX;
    strcpy(direccion.sun_path, "unix_socket");
    unlink(direccion.sun_path);
    servidor_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    ...
}

int main_docente() {
    struct sockaddr_un servidor;
    servidor.sin_family = AF_UNIX;
    strcpy(servidor.sun_path, "unix_socket");
    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    ...
}
```

Ejercicio 2.(25 puntos)

Una ciudad inteligente tiene sensores de calidad del aire distribuidos en distintos barrios, que reportan concentraciones de gases (CO_2 , NO_2 , etc.). Un proceso asociado a cada sensor de la ciudad recopila y preprocesa periódicamente sus datos. Luego, en el centro de monitoreo, un proceso central consolida los datos de los sensores de la ciudad y genera información que se envía a un servidor web donde se mantiene un mapa de contaminación en tiempo real.

Escribir el pseudocódigo que modele este escenario de forma tal que esté libre de deadlock y condiciones de carrera y considerando que:

- Los procesos asociados a sensores conocen la función `preprocDataSensor()` que levanta y preprocesa datos del sensor.

- El proceso central conoce la función `consolidarDataSensores()` que procesa los datos previamente trabajados por los demás procesos, y envía el resultado al servidor web.
- Para optimizar el uso de recursos, el proceso central debe esperar a tener N resultados de los procesos de los sensores antes de consolidarlos para el mapa.
- Por limitaciones de recursos, se pueden ejecutar sólo hasta M ($M < N$) `preprocDataSensor()` en simultáneo.

Ejercicio 3.(25 puntos)

Se tiene un servidor en la nube que ejecuta múltiples procesos pertenecientes a distintos usuarios. Cada usuario puede lanzar varios procesos que complen por el uso de CPU.

El scheduler de este sistema asigna CPU entre usuarios, no entre procesos individuales. Cada usuario se trata como un grupo de procesos, y dentro de cada grupo se establece una política de scheduling interna. Además, existen k categorías de usuarios que determinan su prioridad, donde 0 es el nivel más alto (usuarios Premium) y $k-1$ es el nivel más bajo.

El sistema garantiza que los usuarios usen la CPU de manera justa y evita que un usuario obtenga más procesamiento simplemente lanzando más procesos. El scheduler tiene acceso a métricas que indica el porcentaje de uso de CPU por parte de un usuario en el tiempo reciente.

Responder por verdadero o falso y justificar en ambos casos.

- Todos los usuarios (que son grupos de procesos) deben organizarse en una única cola con política RR con quantum q . Durante el turno de cada usuario, se ejecutarán sus procesos siguiendo su política interna. Cuando termine su turno, se desalojará y pasará al siguiente.
- Todos los usuarios (que son grupos de procesos) deben separarse en colas de prioridad. Para elegir al siguiente usuario dentro de una prioridad, se usa un esquema FIFO.
- Los usuarios de prioridad baja podrían nunca acceder a utilizar el CPU.

Ejercicio 4.(25 puntos)

Considerar el siguiente programa en C:

```
int X[N];
int i = 0;
int a;
int b;
// ...

while (i < N) {
    a = X[i];
    b = X[N - 1 - i];
    X[i] = X[i] + f(a, b);
    i += 3;
}
```

Sabemos que cada página de memoria puede contener exactamente P enteros, la memoria física tiene F marcos de página y sólo cada lectura o escritura a $X[i]$ implica un acceso a memoria.

- Dado $N = 12$, $P = 2$, listar la secuencia de números de página accedidos y justificar cómo se derivan esos números de página a partir de los valores de i . Considerar que el código, los valores de N e i , y la definición de $f()$ se encuentran en una página cargada que no puede desalojarse, y que $X[0]$ se encuentra al inicio de la página 1.
- Usando $F = 3$ marcos de página y la secuencia de pedidos calculada en el inciso anterior, simular los algoritmos de reemplazo LRU y Second Chance. Para cada uno, indicar la cantidad total de fallos de página y el contenido de los marcos tras cada acceso.

1) Empezó haciendo algunas aclaraciones

.) Asumo que en el cliente tengo un arreglo (arreglo-data) de k posiciones. En cada una hay un struct de este tipo

```

typedef struct DATA-T {
    CHAR(8) LU;
    INT NOTA;
} DATA-T;
    
```

Este struct, conocido (redefinición) por el cliente y el servidor

.) El cliente (cliente) conoce también su código de materia, en una variable de Tipo INT llamada CODIGO.

.) Como no recuerdo el funcionamiento de EOF en sockets, decidí usar un struct "especial" que le indique al servidor que es el cliente terminó de agregar notas al acta. ~~Entiendo que no es la manera más elegante de hacerlo, pero funciona.~~ Asumo que las notas son positivas, por lo tanto ninguna nota en ningún DATA-T arreglo-data podrá tener como nota un número negativo. → FUNCIÓN IGUAL QUE EN PIPES

```

//CLIENTE
//OCIENTE
INT MAIN - CLIENTE () {
    //...
    connect (sock);
    WRITE (sock, & CODIGO, sizeof(CODIGO)); // Envío el código de la materia, doctores conocido
    for (int i=0; i<K; i++) {
        WRITE (sock, & (arreglo-data[i]), sizeof(arreglo-data[i]));
    }
    // Envío un DATA-T especial para que el servidor cierre el acta
    CHAR LU(8); // No importa cual...
    DATA-T INVALIDO = { LU, -1 };
    WRITE (sock, & INVALIDO, sizeof(INVALIDO));

    CLOSE (sock);
}
    
```

```

int main-SEVIDOR () {
    //...
    listen (SEVIDOR_Fd, N);
    while (1) {
        client = accept (SEVIDOR_Fd);
        if (fork() == 0) {
            close (SEVIDOR_Fd); // No voy a usar más en el contexto del hijo
            int codigo; // 0 TOMA UN 4-PARAMETRO
            recv (client, &codigo, sizeof (codigo));
            Acta current = openActa (codigo);
            DATA-T DATA-Acta;
            while (1) {
                recv (client, &DATA-Acta, sizeof (DATA-Acta));
                if (DATA-Acta.NOTA == -1) { break; } // El docente termina de enviar
                AÑADIRNOTA (current, &(DATA-Acta.LU), DATA-Acta.NOTA);
            }
            // Se termina del while es porque el docente pidió cerrar el acta
            closeActa (current);
            close (client);
            exit(0);
        } else {
            close (client); // al padre no lo necesita.
        }
    }
}

```

25/25

Pedro Fuentes Urbey
1088/22

44

Año 2

2) // Para Asumo que las siguientes variables están compartidas entre todos los procesos,
 // incluyendo el control. También ^{que} M y N son constantes.

SLOTS = SEM-INIT(M); ✓

MUTEX = SEM-INIT(1); ✓

PROCESADOS = 0; ✓

CONSOLIDAR = SEM-INIT(0); ✓

VOID NODO-CENTRAL () {

while (1) {

CONSOLIDAR-WAIT();

CONSOLIDAR DATA SENSORES (); ✓

}

}

VOID PROCESO-SENSOR () {

while (1) {

SLOTS-WAIT(); ✓

PREPARAR DATA SENSOR (); ✓

SLOTS-SIGNAL (); ✓

MUTEX-WAIT (); ✓

~~Procesar~~

PROCESADOS ++;

if (PROCESADOS == N) {

PROCESADOS
~~PROCESADOS~~ = 0; ✓

CONSOLIDAR-SIGNAL (); ✓

}

MUTEX-SIGNAL (); ✓

}

}

3) b) Vemos que este enunciado es Falso, pues las características planteadas podrían hacer que no se garantice la equidad en la computación. En particular, si usamos FIFO para cada una de las colas de cada prioridad, se podría dar una situación en la que un proceso de alta prioridad quera ejecutar durante mucho tiempo, dejando muchas veces como la cola es FIFO, no se le va a dar control a otro usuario hasta que el actual no termine. Esto ~~se~~ cumple la condición de "El sistema garantiza que los usuarios usen la CPU de manera justa y entre que un usuario obtenga más procesamiento simplemente lanzando más procesos".

Para solucionar esto, podríamos usar RR en cada una de las colas. De este modo, si un usuario lanza más procesos no va a afectar su tiempo de ejecución, pues eventualmente es desalojado y otro usuario toma el control.

Para evitar Starvation en los ~~procesos~~^{usuarios} de menor categoría, se puede implementar feedback, haciendo que un ~~proceso~~^{usuario} pueda salir de categoría. El criterio podría ser el ~~tiempo~~ porcentaje de uso de CPU en el tiempo reciente, y el ~~umbral~~ para salir de categoría podría variar por categoría de usuario y por la cola en la que está actualmente.

Pedro Fariñas Urteaga

1088122

44

Hoja 4

4) a) Vemos que el ~~arreglo de datos~~ arreglo ocupa un total de 6 páginas.
La demanda de páginas recibida en el arreglo es:

i	Páginas accedidas
0	0, 1, 1, 0, 0
3	3, 8, 3, 3
6	6, 5, 6, 6
9	9, 2, 9, 9

Tiempos que:	
$X(0)$	#1
$X(1)$	
$X(2)$	#2
$X(3)$	
$X(4)$	#3
$X(5)$	
$X(6)$	#4
$X(7)$	
$X(8)$	#5
$X(9)$	
$X(10)$	#6
$X(11)$	

luego, eso se traduce en las siguientes páginas:

$i=0$	$i=3$	$i=6$	$i=9$
1, 6, 1, 1, 2, 5, 2, 2	4, 3, 4, 4	5, 2, 5, 5	

b) $F=3$ tamaño de página. Asumo que duplican volúmenes.

LRU (Nº de veces que se repite o la hª. o el tiempo de acceso)								
Página	#1	#2	#3	Orden de lectura	Página	#1	#2	#3
1	1	-	-	1	1 x	1	-	-
6 x	1	6	-	1, 6	6 x	1	6	-
1 ✓	1	6	-	1, 6	1 ✓	1	6	-
1 ✓	1	6	-	1, 6	1 ✓	1	6	-
2 x	1	6	2	1, 6, 2	2 x	1	6	2
5 x	1	9	2	2, 1, 5	5 x	1	5	2
2 ✓	1	5	2	2, 1, 5	2 ✓	1	5	2
2 ✓	1	5	2	2, 1, 5	2 ✓	1	5	2
4 x	4	5	2	5, 2, 4	4 x	4	5	2
3 x	4	3	2	2, 4, 3	3 x	4	3	2
4 ✓	4	3	2	2, 4, 3	4 ✓	4	3	2
4 ✓	4	3	2	2, 4, 3	4 ✓	4	3	2
5 x	4	3	5	4, 3, 5	5 x	4	3	5
2 x	4	2	5	5, 4, 2	2 x	4	2	5
5 ✓	4	2	5	5, 4, 2	5 ✓	4	2	5
5 ✓	4	2	5	5, 4, 2	5 ✓	4	2	5